Relatório do Analisador Sintático (Etapa 2) – GCC130

Giovane Felipe Godoi Oliveira & Luiz Fellipe Resende Lima
Universidade Federal de Lavras (UFLA)
Departamento de Ciência da Computação
Lavras, MG, Brasil

Resumo

Este relatório apresenta o analisador sintático em Bison integrado ao léxico da Etapa 1: a correção solicitada, as decisões de projeto (ambiguidades e tratamento de erros), os resultados da análise da gramática e as dificuldades. A gramática completa em BNF está no Apêndice A.

1 Correção da Etapa 1

O `/*` sem `*/` apenas emitia o erro, e o conteúdo do comentário continuava virando *tokens*. O novo padrão `"/*"([^*]|\*+[^*/])*\**` não aceita `*/` e, portanto, consome o restante do arquivo até o EOF (havendo fechamento, o comentário multilinha casa um caractere a mais e vence). O erro é reportado na linha e coluna de abertura.

2 Integração Flex/Bison

O `parser.y` declara os *tokens* com `%token`; o `bison -d` gera o `parser.tab.h`, incluído no `lexer.l`, e cada regra de *token* passa a fazer `return`. Espaços, comentários e erros léxicos não retornam nada, logo o parser nunca recebe um *token* inválido. O `main` foi movido para o `parser.y` e chama `yyparse()`. No léxico, o antigo `RELOP` foi dividido em `EQOP` (`== !=` <>) e `RELOP` (`< > <= >=`), para que a igualdade tenha precedência menor, como em C. O `compiler.sh` executa `bison`, `flex` e `gcc` e roda o programa.

3 Decisões de Projeto

Estrutura. Um programa é uma sequência de comandos: declaração (`int a, b = 10;`), atribuição, `if`/`else`, `while`, bloco `{ }`, `print(expr);`, `read(id);`, `return expr;` e comando vazio (`;`). Declarações são comandos e podem aparecer dentro de blocos, o que prepara os escopos aninhados da Etapa 3.

Dangling else. Resolvido na própria gramática (Aula 7), sem `%prec`: os comandos são divididos em *casados* (todo `if` possui `else`) e *abertos*. Assim, o `else` sempre se associa ao `if` mais próximo. O `while` foi dividido da mesma forma, pois também termina com um comando.

Precedência e o "-". A ambiguidade das expressões é resolvida por `%left`/`%right`, da menor para a maior: `||` < `&&` < `== !=` <> < `< > <= >=` < `+ -` < `* / %` < `!` < `-` unário, todos associativos à esquerda, exceto os unários. Como decidido na Etapa 1, o léxico gera números sempre sem sinal; o parser trata o `-` como binário (por precedência) ou unário (`MINUS expr %prec UMINUS`). Assim, `x-1` é uma subtração e `-a + b` é `(-a) + b`.

Tratamento de erros. O `yyerror` informa a linha e a coluna do *token* em que o erro foi detectado (ou "fim de arquivo inesperado"). A recuperação usa o modo pânico: `lista_comandos error ';' { yyerrok; }` e `lista_comandos error '}'` descartam *tokens* até um `;` ou `}` e a análise prossegue, reportando vários erros em uma execução. Ao final são exibidos a tabela de símbolos e o total de erros léxicos e sintáticos.

4 Resultados da Análise da Gramática

A gramática possui 45 produções e gerou um autômato LALR(1) com 90 estados e **nenhum conflito** reportado pelo Bison. Os 99 conflitos *shift/reduce* inerentes às expressões ambíguas foram todos resolvidos pelas declarações de precedência (67 como *reduce* e 32 como *shift*); o *dangling else* não gera conflito algum. O `programa.txt` (toda a gramática) é aceito sem erros sintáticos, e o `teste_sintatico_invalido.txt` reporta os seis erros inseridos, por exemplo:

```
syntax error on line 3 and column 7. Input -> "b"
syntax error on line 7 and column 1. Input -> "else"
syntax error on line 12 and column 1. Fim de arquivo inesperado.
```

5 Dificuldades Encontradas

(i) Eliminar o *dangling else* sem conflitos exigiu reestruturar todos os comandos em casados/abertos, inclusive o `while`. (ii) O lexer soma o tamanho do lexema à coluna antes do `return`; para o erro apontar o início do *token*, a posição passou a ser guardada antes desse avanço. (iii) A regra `error '}'` pode consumir o `}` de um bloco (ex.: `{ a = 1 }`), gerando um erro em cascata no fim do arquivo.

A Gramática Completa em BNF

```
<programa>   ::= <lista_cmd>
<lista_cmd>  ::= <lista_cmd> <comando> | (vazio)
               | <lista_cmd> error ";"
               | <lista_cmd> error "}"
<comando>    ::= <casado> | <aberto>
<casado>     ::= "if" "(" <expr> ")" <casado>
                     "else" <casado>
               | "while" "(" <expr> ")" <casado>
               | <simples>
<aberto>     ::= "if" "(" <expr> ")" <comando>
               | "if" "(" <expr> ")" <casado>
                   "else" <aberto>
               | "while" "(" <expr> ")" <aberto>
<simples>    ::= <decl> | <atrib> | <bloco> | <print>
               | <read> | <return> | ";"
<decl>       ::= <tipo> <lista_decl> ";"
<tipo>       ::= "int" | "float"
<lista_decl> ::= <item_decl>
               | <lista_decl> "," <item_decl>
<item_decl>  ::= ID | ID "=" <expr>
<atrib>      ::= ID "=" <expr> ";"
<bloco>      ::= "{" <lista_cmd> "}"
<print>      ::= "print" "(" <expr> ")" ";"
<read>       ::= "read" "(" ID ")" ";"
<return>     ::= "return" <expr> ";"
<expr>       ::= <expr> "||" <expr>
               | <expr> "&&" <expr>
               | <expr> <op_ig> <expr>
               | <expr> <op_rel> <expr>
               | <expr> "+" <expr> | <expr> "-" <expr>
               | <expr> "*" <expr> | <expr> "/" <expr>
               | <expr> "%" <expr>
               | "!" <expr> | "-" <expr>
               | "(" <expr> ")"
               | ID | NUM_INT | NUM_FLOAT
<op_ig>      ::= "==" | "!=" | "<>"
<op_rel>     ::= "<" | ">" | "<=" | ">="
```

`ID`, `NUM_INT` e `NUM_FLOAT` são definidos no léxico (Etapa 1). A precedência dos operadores é a descrita na Seção 3.